Manual Tests and BO Code Changes

16 October 2026

1 Introduction

Previous document is 'Modifications To One Shape BO Package on 14 October 2026'. ALso see relevant .txt files in Code, Topology Optimisation, Bayesian Optimisation Part3 261008 Package Plan:

- `261015_Modifications_Proposed`
- `261015_Codex_Prompts`: elaboration on the above... Seems rather over complex
- `261015_Manual_Prompts`
- `261016_Modification_Implementation`: the relevant commits and what changed and why.

This will provide guidance and documentation on the manual tests performed.

2 Numerical Stability Study

Necessary to determine the observation-noise variance. See section 6.1.1-6.1.3 in the previous report. What may be happening in seed 76 case (see `Triangle_1` folder) is that the inferred noise is too great. Really, this should be close to zero (the observation noise should be constant - i.e. the same rasterised shape should give the same FE result, as this is not stochastic). Only the signal noise should vary (i.e. variation in the compliance result from `y_train`).

See `https://botorch.org/docs/models`

Currently the noise is homoskedastic (noise is not provided as an input, instead it is inferred). Now the noise becomes fixed (noise is provided as an input). What should that fixed value be? That is the purpose of this offline study.

2.1 Control

In the config, require:

```
observation_noise_variance = None
warm_start_policy = None
```

Use cold fits for the main comparison. Warm-starting would otherwise give different candidates different inherited hyperparameters. Currently, the GP always undergoes a warm start. Need to modify code accordingly (and check correct implementation).

For every comparison, hold constant:

- Identical archived training
- Matern kernel and priors
- Outcome transform
- Fitting optimiser
- device/dtype

- Probe and held-out points
- Initialisation or initialisation seed

2.2 Important variance-unit correction

Currently, `observation_noise_variance` is supplied in pre-standardization response units, where $y = -log(\text{compliance})$. BoTorch's `Standardize` transform converts it internally as:

$$\nu_{model} = \frac{\nu_{config}}{s_y^2} \tag{1}$$

So to test a desired standardized variance $\nu*$ at a fixed training snapshot, supply:

$$\nu_{config} = \nu_{model} * s_y^2 \tag{2}$$

BoTorch applies the same outcome transform to Y and $Yvar$, including division of variance by the squared standard deviation. Additionally, the installed GPyTorch floors double-precision fixed noise at 10^{-6}, thus cannot use proposed 10^{-8}.

To check the installed floor:

```
python - <<'PY'
import torch, botorch, gpytorch
from gpytorch.settings import min_fixed_noise

print("BoTorch:", botorch.__version__)
print("GPyTorch:", gpytorch.__version__)
print("double fixed-noise floor:", min_fixed_noise.value(torch.double))
PY
```

The comparison should be run on **fixed offline snapshots**. Compare to a baseline run, e.g. `M0_Verify_Reproducible_Outputs_Seed76` below (although need to re-run this with cold-starting as previous subsection requires).

```
python - <<'PY'
from one_shape_bo import reconstruct_training_set

run = "outputs/manual_tests/M0_Verify_Reproducible_Outputs_Seed76"
data = reconstruct_training_set(run, 14, position="before")
y = data["gp_response_prestandardization"]
sample_variance = y.var(correction=1).item()

print("response variance:", sample_variance)
for standardized_variance in (1e-6, 1e-5, 1e-4):
    config_variance = standardized_variance * sample_variance
    effective = max(standardized_variance, 1e-6)
    print(
        "requested standardized:", standardized_variance,
        "config variance:", config_variance,
        "effective standardized:", effective,
    )
PY
```

Importantly, **a single constant config value cannot remain a constant standardised variance throughout an evolving BO run** because s_y^2 (the sample variance) changes. That is another reason to perform the comparison on **fixed offline snapshots**.

2.3 Study Size

For three holes, the dimension is 21 and the default burn-in is 105. Retain that throughout:

- Burn-in: 105
- BO evaluations: 30-50
- Restart, sampling, kernel, and physical settings: intended production defaults
- Device: CPU

2.4 Hold-Out

Held-out evaluations are recorded designs whose X and compliance are deliberately excluded from GP fitting and then used to test predictions. e.g.:

- Fit using burn-in plus BO evaluations 1-5
- For each learned/fixed-noise vandidate, predict the already recorded designs from BO evaluations 6-15
- Compare predictions with their recorded $-log($**compliance** responses

The requested/config/effective variance conversion is calculated once for each training snapshot because its response standard deviation changes. The separate representative probe set needs no FEM values; it is used only to measure posterior-mean and posterior-SD variation.

2.5 Seeds and Initialisations

- Pilot: 5 seeds
- Fits per snapshot/candidate: 3 initialisations.
  - Snapshot: one fixed training cutoff e.g. burn-in plus BO 1-5
  - Candidate: learned noise or one fixed effective variance
  - Initialisation: Fit that exact model/data combination three times from three different starting hyperparameter states

Use the same three initialization states for every noise candidate. Merely changing `torch.manual_seed` may not change the package's default GP initialisation, so verify that initial hyperparameters actually differ. A rigorous multi-initialisation study needs a stand-alone analysis script/notebook that constructs the same `MixedSingleTaskGP`, perturbs valid raw hyperparameters, and then calls the same `fit_gpytorch_mll`.

2.6 Implementation

See the new folder in src named `src/one_shape_bo/numerical_noise_study.py`. This allows for an automated numerical study, which finally selects the best observation noise. Note that this always enforces cold-start GPs (rather than the conventional script with always warm-start, unless warning, then perhaps take action).

2.6.1 Study Design

- Seeds: 0, 1, 2, 3, 76 (76 originally triggered strange behaviour, doesn't seem to be the case anymore, might replace with '4').
- Standardized fixed variances: 1e-6, 1e-5, 1e-4
- Control: learned noise
- Initialisations: 3 (expect the same result as observation noise should be near-zero as FEM is deterministic)
- Snaphsots: After BO evaluations 0, 5, 10, 20 (where 0 means only the burn-in)
- Held-out window: next 10 BO evaluations
- Required archive: 105 burn-in plus at least 30 BO evalulations per seed

2.6.2 Controlled Fitting

For every seed and snapshot, load the identical training X/Y from `evaluation_archive.pt` and reserve the next ten recorded BO evaluations as held-out data. Calculate the training response variance s_y^2 and convert each requested standardised variance (as explained). Construct the same `MixedSingleTaskGP` and associated priors and transforms (this is always a cold-started hyperparameter GP, unlike current implementation). Apply one of three paired cold initiaisations:

- Initialisation 0: package default
- Initialisations 1-2: reproducible perturbations of unconstrained mean/kernel parameters
- The same perturbations are used for every noise candidate in that cell

Call `fit_gpytorch_mll` with `max_attempts=1`, preventing BoTorch retries from replacing the explicitly controlled initialisation. Capture every warning or failure and continue to the next fit.

2.7 Terminal Steps

2.7.1 Create Environment and set CPU

```
cd /d "C:\path\to\one_shape_bo"
dir
conda env create -f environment.yml
conda activate one-shape-bo
python -m pip install -e .
set CUDA_VISIBLE_DEVICES=-1
python -c "import one_shape_bo; from one_shape_bo.runtime import DEVICE; print(one_shape_bo.__file__); print(DEVICE)"
```

2.7.2 Run As Normal

The source BO runs may remain warm-started. The numerical study does not reuse their fitted GP states or hyper parameters- it extracts only the archived design vectors and compliance values, then independently cold-starts every study fit. Thus:

```
ptyhon
from pathlib import Path
from one_shape_bo import OptimizationConfig, run_optimization

base = Path("outputs/Manual_Tests")
base.mkdir(parents=True, exist_ok=True)

seeds = (0, 15, 23, 46, 53)

for seed in seeds:
    output = base / f"Seed_{seed}"

    config = OptimizationConfig(
        seed=seed,
        iterations=30,
        local_start_strategy="archive",
        output_dir=output,
        observation_noise_variance=None,
    )

    print(f"Starting seed {seed}: {output}")
    result = run_optimization(config)
    print(f"Completed seed {seed}")
```

You'll need to manually indent within the terminal. This uses the default three-hole problem, and default burn-in of 105 (expect it to take some time to evaluate all 5 requested seeds).

2.8 Verify Runs

```
required = ("run_config.json", "run_manifest.json", "evaluation_archive.pt",)
for seed in seeds:
    folder = base / f"Seed_{seed}"
    print(seed, {name: (folder / name).exists() for name in required})
```

All values should be True.

2.9 Run the Study

```
from one_shape_bo.numerical_noise_study import (
    run_numerical_noise_study,
    summarize_numerical_noise_study,
    decide_provisional_fixed_variance,
)

runs = {
    seed: base / f"Seed_{seed}"
    for seed in seeds
}

study_dir = base / "numerical_noise_study"

fit_records = run_numerical_noise_study(
    run_directories=runs,
    output_directory=study_dir,
    standardized_variances=(1e-6, 1e-5, 1e-4),
    fit_after=(0, 5, 10, 20),
    holdout_size=10,
    initializations=3,
)

summary = summarize_numerical_noise_study(
    study_dir,
    make_plots=True,
)

decision = decide_provisional_fixed_variance(study_dir)

print("Fits:", len(fit_records))
print("Decision:", decision["status"])
print("Variance:", decision["selected_standardized_noise_variance"])
print("Rationale:", decision["rationale"])
```

This performs 240 GP fits: 5 seeds x 4 snapshots x 4 noise models x 3 initializations. The decision proposes the best observational noise to fix.

Once complete, revise main code so that the decision is implemented: the GP always has this best observational noise (i.e. it is fixed), and never learns it from the data (i.e. what is currently implemented: homoskedastic noise).

3 Study Results

See `C:\Users\pemb6626\OneDrive-Nexus365\Documents\POSTDOC\Code\Topology_Optimisation\Bayesian_Optimisation Part3\261008_Package_Plan\outputs\Manual_Tests`.

Notice that seed 23 exhibits many health warnings in the output log and indeed the compliance has a large fluctuation. Is there a way to establish how the proposed fixed variance addresses this or resolves this?

After running the specific config as instructed above:

```

>>> required = ("run_config.json", "run_manifest.json", "evaluation_archive.pt",)
>>> for seed in seeds:
... folder = base / f"Seed_{seed}"
... print(seed, {name: (folder / name).exists() for name in required})
...
0 {'run_config.json': True, 'run_manifest.json': True, 'evaluation_archive.pt': True}
15 {'run_config.json': True, 'run_manifest.json': True, 'evaluation_archive.pt': True}
23 {'run_config.json': True, 'run_manifest.json': True, 'evaluation_archive.pt': True}
46 {'run_config.json': True, 'run_manifest.json': True, 'evaluation_archive.pt': True}
53 {'run_config.json': True, 'run_manifest.json': True, 'evaluation_archive.pt': True}
>>> from one_shape_bo.numerical_noise_study import (run_numerical_noise_study, summarize_numerical_noise_study,
      decide_provisional_fixed_variance,)
>>> runs = {seed: base/ f"Seed_{seed}" for seed in seeds}
>>> study_dir = base / "numerical_noise_study"
>>> fit_records = run_numerical_noise_study(run_directories=runs, output_directory=study_dir, standardized_variances=(1e-6, 1e-5,
      1e-4), fit_after=(0, 5, 10, 20), holdout_size=10, initializations=3,)
>>> summary = summarize_numerical_noise_study(study_dir, make_plots=True,)
>>> decision = decide_provisional_fixed_variance(study_dir)
>>> print("Fits:", len(fit_records))
Fits: 240
>>> print("Decision:", decision["status"])
Decision: provisional_selection
>>> print("Variance:", decision["selected_standardized_noise_variance"])
Variance: 1e-05
```

```
>>> print("Rationale:", decision["rationale"])
Rationale: Eligible candidates were compared by paired held-out NLPD. Within one standard error of the best, warning rate and
     initialization sensitivity were preferred; larger variance was used only as the final regularising tie-break.
>>>
```

Thus, 1e-05 is the decided fixed variance.

4 Health Monitoring Study

The current numerical-noise study can inform several health thresholds, but it cannot establish all of them rigorously because its fits are independent cold fits using a fixed archived probe set. Production health monitoring observes sequential, usually warm-started fits and iteration-specific restart pools.

4.1 Thresholds this study can inform

Configuration input	Evidence available	Usefulness
`health_nu_min`	`signal_fraction_nu` for learned-noise fits	Useful for replacing or validating the heuristic `0.01`
`health_signal_variance_min`	`signal_variance` for fixed-noise fits	Useful after choosing the provisional fixed variance
`health_mean_spread_min`	`probe_mean_spread_ratio`	Useful as an initial threshold
`health_posterior_std_min`	`probe_latent_sd_ratio_q90`	Useful as an initial threshold
`health_predictive_z_max`	Held-out standardized residual metrics	Only approximate
`health_predictive_window`	Not studied directly	Requires sequential replay

A threshold should not simply be the smallest observed value. Instead, classify each fit independently as acceptable or unhealthy using:

- fitting success and numerical warnings;
- held-out RMSE and negative log predictive density;
- predictive coverage;
- sensitivity to the three initializations;
- posterior mean and uncertainty collapse.

Then determine which health-metric values discriminate poorly behaving fits from acceptable fits across held-out seeds.

For example, `health_signal_variance_min` should be placed above signal variances consistently associated with collapsed predictions or poor held-out agreement, but below the normal range of stable fits.

4.2 Why a second study is required

Two mismatches prevent the current study from producing final production thresholds:

1. Its GP fits are always independently cold-started. Production normally warm-starts sequentially.
2. Its probe set is the fixed union of archived feasible designs. Production computes mean-spread and posterior-SD metrics over the current iteration's global/local restart pools.

The held-out predictions are also block predictions: one snapshot predicts several future observations without updating the GP. Production predictive health is a rolling sequence of one-step-ahead residuals, refitting after every new evaluation.

Therefore, the current study should select the provisional fixed variance and provide preliminary threshold ranges. A separate sequential replay study should establish the final runtime thresholds.

4.3 Proposed health-threshold study

Add a standalone `health_threshold_study.py` that reuses archived evaluations and performs no FEM calculations. For each of the five seeds:

1. Replay every archived BO iteration in order.
2. Fit both:
   - learned-noise control;
   - provisionally selected fixed noise.
3. Reproduce production initialization:
   - first fit cold;
   - subsequent fits warm;
   - optionally fit an independent cold reference alongside each warm fit.
4. Before adding evaluation t, predict its recorded response.
5. After observing it, append it and continue to $t + 1$.
6. Record:
   - signal variance, noise variance, SNR and ν;
   - posterior mean-spread ratio;
   - posterior-SD ratio;
   - one-step standardized residual;
   - rolling RMS residual for candidate windows such as 3, 5 and 10;
   - fit warnings/failures;
   - held-out error and predictive density;
   - warm-versus-cold fitted hyperparameter divergence.

For thresholds to transfer exactly to production, one of these is required:

- **Preferred:** persist the representative restart-pool points used by each production fit as a compact tensor reference.
- **Alternative:** define a deterministic, fixed feasible probe set and use it in both production health monitoring and the threshold study.

The first option preserves current production definitions and requires only additional logging; it does not require additional FEM evaluation.

4.4 Threshold calibration

Use seed-level validation rather than pooling every iteration indiscriminately:

1. Hold one seed out.
2. Use the other four seeds to test a grid of thresholds.
3. Select thresholds that detect independently defined unhealthy fits while limiting warnings on stable fits.
4. Test those thresholds on the held-out seed.
5. Repeat for every held-out seed.
6. Report the resulting threshold range and whether one common value is stable across seeds.

Because all warning conditions are joined by logical OR, calibrate the complete warning rule as well as each threshold separately. Otherwise, several individually reasonable thresholds could produce an excessive combined warning rate.

For `health_predictive_z_max`, evaluate each proposed window size. If standardized residuals were ideally normal, then approximately

$$k\,\mathrm{RMS}(z)^2 \sim \chi^2_k,$$

so a statistical starting threshold is

$$\sqrt{\frac{\chi^2_{k,1-\alpha}}{k}}.$$

However, the final value should be checked empirically because BO observations are adaptive and the GP may be misspecified.

4.5 Proposed outputs

The new study should produce:

- `gp_health_threshold_study.csv`;
- `threshold_candidates.csv`;
- `cross_seed_validation.csv`;
- plots of each metric against predictive failure and fit instability;
- `recommended_health_config.json`;
- `no_recommendation` for any threshold not supported consistently across seeds.

A proposed JSON result might be:

```
[language={},basicstyle=\ttfamily\small,breaklines=true]
{
  "noise_mode": "prescribed",
  "observation_noise_variance_standardized": 0.00001,
  "health_nu_min": null,
  "health_signal_variance_min": 0.0,
  "health_mean_spread_min": 0.0,
  "health_posterior_std_min": 0.0,
  "health_predictive_z_max": 0.0,
  "health_predictive_window": 5,
  "status": "provisional"
}
```

The zeros above are placeholders—the study must determine them.

In summary: use the current numerical-noise study to choose the provisional fixed variance and identify preliminary collapse ranges. A sequential, production-matched, cross-seed replay study is required to establish defensible final `health_*` settings, particularly the predictive threshold and window.

4.6 WSL or Conda Prompts

Once the numerical study is complete, run the replay:

```
python
from pathlib import Path
from one_shape_bo.health_monitoring_study import (run_health_monitoring_study, summarize_health_monitoring_study,
    calibrate_health_thresholds,)
numerical_dir = Path("outputs/Manual_Tests/numerical_noise_study")
health_dir = Path("outputs/Manual_Tests/health_monitoring_study")
replay_records = run_health_monitoring_study(numerical_study_directory=numerical_dir,output_directory=health_dir,windows=(3, 5, 10),)
```

If `decision.json` does not exist, this automatically runs the numerical study's decision function. It stops safely if no fixed variance was selected.

Next, generate summaries and plots:

```
summary = summarize_health_monitoring_study(health_dir, make_plots=True)
import json
print(json.dumps(recommendation, indent=2))
```

And calibrate the thresholds:

```
recommendation = calibrate_health_thresholds(
    health_study_dir,
    adverse_z_threshold=2.576,
    maximum_false_positive_rate=0.10,
)

print(json.dumps(recommendation, indent=2))
```

The main result is saved at:

```
print(health_study_dir / "recommended_health_config.json")
```

The directly applicable, non-null recommendations are available with:

```
print(
    json.dumps(
        recommendation["optimization_config_overrides"],
        indent=2,
    )
)
```

To override the numerical study's decision (e.g. to examine 10^{-6}), use:

```
    replay_records = run_health_monitoring_study(
    numerical_study_directory=numerical_study_dir,
    output_directory=Path(
        "outputs/Manual_Tests/health_monitoring_study_1e-6"
    ),
    selected_standardized_variance=1e-6,
    windows=(3, 5, 10),
)
```

For five seeds with 30 BO evaluations, the replay performs approximately 295 GP fits: one initial fit and warm/cold pairs thereafter. It performs no new physical evaluations. Note that `health_nu_min` will be marked not applicable for prescribed noise—the runtime uses `health_signal_variance_min` in that mode. Any threshold lacking consistent cross-seed evidence remains null rather than receiving an invented default.

5 Restart-budget and proportion study

This study is necessary to determine the correct balance of global and local restarts (proportion and total). This should be conducted fairly, so use nested restarts and appropriate random seed streams.

The configuration will change. The following is added:

```
adaptive_restart_allocation: bool = True
restart_source_shortfall_policy: str = "substitute"
```

With `adaptive_restart_allication = False` would hold the requested `ng_rst` and `nl_rst` constant. A separate total is unnecessary because the fixed total is unambiguously `ng_rst` + `nl_rst`.

Production diagnostics should distinguish:

- Requested global/local starts
- Available global/local starts
- Actually attempted global/local restarts
- Successful optimised endpoints by source
- duplicate and infeasible endpoints
- Source substitutions and quota shortfuls

The following will be output:

- `candidate_pools.pt`
- `restart_optimizations.pt`
- `restart_trials.csv`
- `allocation_results.csv`
- `summary_by_allocation.csv`
- `study_config.json`
- `study_manifest.json`
- `recommended_restart_allocation.json`

The recommendation should use paired seed-and-snapshot comparisons, select the smallest budget within one standard error of the best mean feasible LogEI, then choose the proportion with the best paired performance and acceptable shortfall/failure rate. This is an acquisition-optimization recommendation, not proof of better final compliance. The top two or three allocations should subsequently be compared using full BO runs across the same five seeds.

5.1 Proposed Python Prompts

```
cd /d "C:\path\to\one_shape_bo"
conda activate one-shape-bo
python
```

Conduct a two-start smoke test

```
from pathlib import Path

from one_shape_bo.restart_budget_study import (
    run_restart_budget_study,
    summarize_restart_budget_study,
    recommend_restart_allocation,
)

source = Path("outputs/manual_tests/M1_Numerical_Noise_Study")

```

```
smoke_dir = Path(
    "outputs/manual_tests/M4_Restart_Budget_Study/smoke"
)

smoke_rows = run_restart_budget_study(
    run_directories={
        0: source / "Seed_0",
    },
    numerical_study_directory=source / "numerical_noise_study",
    output_directory=smoke_dir,
    fit_after=(0,),
    allocations=((1, 0), (0, 1), (1, 1)),
    initialization_id=0,
    raw_pool_multiplier=3,
    optimizer_maxiter=250,
    optimizer_timeout_seconds=60.0,
    expected_seed_count=1,
)

print("Smoke results:", len(smoke_rows))
print("Output:", smoke_dir.resolve())
```

This optimizes only one global and one archive start. Its worst-case optimizer time is approximately two minutes. It validates setup but provides no scientific conclusion.

Then, in a fresh python process:

```
from pathlib import Path

from one_shape_bo.restart_budget_study import (
    run_restart_budget_study,
    summarize_restart_budget_study,
    recommend_restart_allocation,
)

source = Path("outputs/manual_tests/M1_Numerical_Noise_Study")

runs = {
    seed: source / f"Seed_{seed}"
    for seed in (0, 15, 23, 46, 53)
}

full_dir = Path(
    "outputs/manual_tests/M4_Restart_Budget_Study/full"
)

full_allocations = (
    (0, 5), (1, 4), (3, 2), (4, 1), (5, 0),
    (0, 10), (2, 8), (5, 5), (8, 2), (10, 0),
    (0, 15), (4, 11), (8, 7), (10, 5), (11, 4), (15, 0),
    (0, 20), (5, 15), (10, 10), (15, 5), (20, 0),
    (0, 30), (8, 22), (15, 15), (20, 10), (22, 8), (30, 0),
)

full_rows = run_restart_budget_study(
    run_directories=runs,
    numerical_study_directory=source / "numerical_noise_study",
    output_directory=full_dir,
    fit_after=(0, 5, 10, 20),
    allocations=full_allocations,
    initialization_id=0,
    raw_pool_multiplier=3,
    optimizer_maxiter=250,
    optimizer_timeout_seconds=60.0,
)

summary = summarize_restart_budget_study(full_dir)
recommendation = recommend_restart_allocation(full_dir)

print("Allocation results:", len(full_rows))
print("Recommendation:")
print(recommendation)
print("Output:", full_dir.resolve())
```

6 Seed 23 Investigation

See the corresponding folder in outputs. In stages, varied the following controls:

- Whether a cold restart is performed after GP health warning
- Whether the new greedy global starts are considered, or continue with the static rank
- Impose the overrides proposed from the `numerical_noise_study` and the `health_monitoring_study`

6.1 Investigation and Questions

Here is a report (with directions for Codex) on the investigation, recorded within `M3_Seed_23_Investigation`:

This study is conducted on the code at commit 74f3b8b.

`numerical_noise_study` (seeds 0, 15, 23, 46, 53) and `health_monitoring_study` complete. Results in `outputs/manual_tests`. Notice that seed 23 flags multiple GP health warnings. The GP becomes effectively uninformative, similar to previous Seed 76 outputs.

Then, prescribed the `optimization_config_overrides` proposed in the `recommended_health_config.json`. Conducted the following "study": intended examination.

(1) `Seed_23_v2`: always warm-start, `global_restart_selection_strategy` set to "`greedy_dynamic_novelty`", use the `optimization_config_overrides`. This should be compared to `Seed_23_v3`. Hope to see faster tendency towards small compliance. Observation: Earlier GP Health Warnings again (see below - is this because applied more stringent results from the health monitoring study?). Indeed, much faster tendency towards small compliance; 121 after iteration 9 compared to 134 in `Seed_23_v3` (and 141 in `Seed_23`). Final compliance is 112; after only 30 iterations, see $\tilde{2}0\%$ improvement and no adverse GP behaviour. GP surrogate seems to be informative.

(2) `Seed_23_v3`: always warm-start, `global_restart_selection_strategy` set to "`static_rank`", use the `optimization_config_overrides`. This should have been conducted before v2. Intend to compare to `Seed_23` (within `M1_Numerical_Noise_Study`). Hope to see fewer GP health warnings. Hope that GP remains informative. Comparison observation: Earlier GP Health Warnings (is this because applied more stringent results from the health monitoring study?). GP has less fluctuation, and already finds better result after iteration 6. No spike in compliance at iteration 9.

(3) `Seed_23_v4`: cold start if GP health warning flagged (`warm_start_policy = health_reset`), `global_restart_selection_strategy` set to "`greedy_dynamic_novelty`", use the `optimization_config_overrides`. Hope to see even faster tendency towards minimum than v2. Hope for less variation in compliance. Observation: the cold fitting doesn't seem to be working? GP Health Warning is flagged after the 2nd iteration. Claim to cold fit the next iteration. Result from iteration 3 (and 4) is identical to iteration 3, v2 - no evidence of cold restart (when v2 had warm restart). Result from iteration 5 diverges to v2 (why, when the GP parameters should be warm-started between iteration 4 and 5?). Final result is worse than v2. The final result found in v4 is very similar to the final result found in v3. This suggests that cold-fitting after GP health warning is not advantageous?

You must: 1. Inspect the `numerical_noise_study` (`M1_Numerical_Noise_Study`) and `health_monitoring_study` (`M2_Health_Monitoring_Study`) outputs. Were these conducted correctly? Are the findings reasonable? 2. Inspect the `Seed_23` exam (`M3_Seed_23_Investigation`, above report, but you should also examine the relevant folders). What can be concluded?

Be concise. Plan mode/reporting to the chat only. Follow AGENTS.md.

6.2 Response and Next Steps

M1 was executed correctly according to its design. It completed 240 independent fits: five seeds × four snapshots × four noise models × three cold initialisations, with no failed fits. The provisional choice of standardized variance 10^{-5} is reasonable, but weakly distinguished:

- 10^{-6} generated fitting warnings in 40
- 10^{-5} and 10^{-4} had almost identical held-out RMSE and NLPD.
- 10^{-5} won mainly through slightly lower initialisation sensitivity—not clearly superior prediction.

It should therefore remain a provisional choice, as stated in [decision.json](`/home/eleno/projects/one_shape_bo/outputs/manual_tests/M1_Numerical_Noise_Study/numerical_noise_study/decision.json`). M2 also completed correctly computationally: 150 successful replay fits, production-style warm starts, independent cold references, and leave-one-seed-out calculations. However, its recommended thresholds are not yet robust enough for automatic intervention:

- Signal-variance validation detected only about 13% of adverse cases on average.
- Posterior-SD validation detected about 18%.
- Some held-out folds exceeded the nominal 10% false-positive limit substantially.
- The three-point predictive residual performed best, but remained inconsistent between seeds.
- Only five independent seed trajectories were available; successive iterations within a seed are correlated.

Thus the thresholds in [`recommended_health_config.json`](`/home/eleno/projects/one_shape_bo/outputs/manual_tes
are reasonable exploratory warning thresholds, but the evidence does not support treating every warning as proof that the GP is uninformative or as a reliable trigger for cold fitting. There is also an important units error in their application to M3:

- M1 selected 10^{-5} on the standardized GP-output scale.
- `OptimizationConfig.observation_noise_variance` expects variance in raw pre-standardisation -log(compliance) units.
- M3 supplied raw 1e-5. Its effective standardized noise was approximately 6.9×10^{-5} to 1.05×10^{-4}, around 7–10.5 times the studied value.

Consequently, the M2 thresholds and M3 GP were not operating under the same fixed-noise model.

Run	Difference	Best Compliance
M1, Seed 23	Static selector, learned noise	141.121
v2	Greedy selector, always warm	112.989
v3	Static selector, always warm	125.7
v4	Greedy selector, health reset	125.199

Table 1: Comparison of same seed with different parameters

And in the M1 example, the GP clearly degenerates. Conclusions:

- v2 versus v3: This seed strongly favours greedy dynamic novelty—about 10% lower best compliance—but one seed cannot establish general superiority. This is the only relatively clean M3 comparison because v2 and v3 otherwise share their configuration and source state.
- M1 Seed 23 versus v3: This does not isolate the selector or health monitoring. It simultaneously changes learned noise to prescribed noise, applies health thresholds, and uses a different recorded source state.

- Early warnings: They are largely posterior-SD threshold crossings. v2 raised 23 warnings and v3 25, despite v2 producing an excellent result. Therefore these warnings do not, by themselves, demonstrate an uninformative surrogate.
- Cold reset is working. In v4, fit 3 raised the first warning and recorded `next_fit_forced_cold`; fit 4 was cold. Further warnings caused fits 7–30 to be cold. This is explicitly recorded in [`gp_fits.csv`]
- v4 can still select the same point immediately after a reset because cold and warm hyperparameter optimisation may converge sufficiently closely to give the same acquisition winner.
- Divergence at iteration 5 is expected: although fit 5 was warm-started, v4 inherited the state of its cold fit 4, whereas v2 inherited its warm fit 4.
- v4's worse result does not prove cold fitting is generally harmful. It shows that this aggressive threshold/reset combination—26 warnings and 26 cold fits including the initial fit—was inferior on this single trajectory.
- The apparent similarity between v3 and v4 is numerical coincidence, not evidence that static ranking and health resetting are equivalent.

Overall, M3 supports further multi-seed testing of the greedy selector, but it does not validate `health_reset`. That policy should remain experimental until the standardized-versus-raw noise mismatch is resolved and the thresholds show materially stronger cross-seed discrimination. No files were changed.

7 Restart-budget and proportion study: Outcomes

See `M4_Restart_Budget_Study\full` in the `manual_tests` outputs folder. Recommended 'provisional' result is 4 global and 11 local restarts. However!

A reasonable provisional conclusion is:

- Use approximately 15 total restarts, biased toward archive/local starts.
- The evidence supports 5 global + 10 local just as strongly as the reported 4 global + 11 local.
- It does not establish that 4/11 is uniquely optimal.

The recommendation selected 4/11, but 4/11 and 5/10 found effectively identical acquisition optima in all 20 seed/snapshot comparisons. The tiny difference in their mean regret is numerical noise (10^{-13}); 5/10 was marginally faster.

Thus:

- 5G/10L is the defensible balanced recommendation.
- 0G/10L is the speed-oriented option: about 22% faster than 5G/10L and identical in 17/20 cases, but it missed appreciably in three cases and eliminates global exploration.
- 10G/10L is the conservative acquisition-optimization option: approximately 23% slower than 5G/10L, for only one additional effectively exact block.
- 1G/4L is not merely narrowly worse: 5G/10L beat it in 8/20 blocks, sometimes substantially, while matching it in the other 12.
- On the study CPU, choosing 5G/10L rather than 1G/4L costs about 29 seconds per BO iteration—roughly 15 extra minutes over 30 iterations or 4.3 hours over 525 iterations. These estimates exclude FEM, GP fitting and pool generation.

Caveats The conclusion is limited because:

- Only five seed trajectories and four correlated snapshots per seed were studied.
- The metric is frozen-GP feasible LogEI, not final compliance; no counterfactual FEM evaluations occurred.

- "Local" means archive-derived starts optimized over the full box, not radius-constrained local optimization.
- Results depend on the selected fixed-noise GP, candidate ordering, weights, pool multiplier and optimizer settings.
- The 15-restart recommendation is driven by 19/20 exact blocks and one miss; another seed set could move the apparent knee.
- Production source substitution and adaptive allocation were disabled in this controlled study.
- Timings are hardware-specific.

Therefore, the study identifies a convincing acquisition-optimization knee around 15 archive-heavy restarts, but final adoption of 5G/10L should depend on sequential multi-seed BO demonstrating equal or better final compliance. No files were changed.

Perhaps next should run full studies with the best performing local/global splits? e.g. three seeds and compare results? Or could simply progress with 5G/10L... Incidentally this is the opposite of the current default split of 10G/5L... Will 5G/10L allow sufficient exploration over the complete run?

8 Week 19th October Tasks

- Read the above latest section for the response to `M3_Seed_23_Investigation`. Once understood, ask Codex what must change/ be conducted next (elaborate on this, according to understanding of above response)? Did I apply the incorrect fixed observation noise? Conduct investigation again? Remember: perfection is the enemy to progress. My conclusions may already be sufficient, particularly as no longer see unstable GP for seed 76. Perhaps simply proceed with decision (although provisional)... How to make it unprovisional?
- Analyse the results from `M4_Restart_Budget_Study` and paste results from the desktop into home folder. What do these show: can I make a conclusion on best local/global budget? At what sacrifice to computational speed (e.g. does the best exhibit a substantial enough improvement to a result with fewer restarts)?
- Make a conclusion on the defaults, from the studies up to M4, e.g. the appropriate observation noise, whether a cold start should be performed after a health trigger, what the `nl_rst` vs `ng_rst` budget should be.
- Next: final study - need to investigate the novelty vs acquisition weighting. This is the final triangular study
- Then, choose the final defaults, carefully impose (check that correct with Codex) and run for e.g. 10 seeds and compare the output (full BO optimisation study).
- Only if happy with results, may progress with the ellipse changes
- Meanwhile, could investigate having larger spacing between triangles so that this can be imposed with ellipses later
- End of week/over weekend run for multiple ellipse seeds with the desired dimensions and loading. Could also examine a full beam (will take longer - larger burn-in and more runs) (option).